Matchcall: A Fact-Grounded, Low-Cost Pipeline for Spoken Football Commentary

Aryaman

Supervisor: Nabin Sir

Department of Computer Science, [University]

27 October 2026

ABSTRACT

Automatic football commentary must detect events, identify players and teams, decide what is worth saying, generate grounded language, and voice it in time. Recent systems apply large multimodal models, but they are expensive, difficult to reproduce on consumer hardware, and can hallucinate facts. We present Matchcall, a modular pipeline that keeps deterministic facts separate from surface realisation. A score-bug reader supplies goal anchors; a motion-gated window scan and a ResNet-18 classifier propose other events; YOLO26n with ByteTrack, jersey clustering and possession estimation supply player-level facts; GPT-2 with LoRA generates commentary under a grounding validator with a template fallback; Fish Audio synthesises speech. On a 64.5-minute ARG–FRA broadcast, the pipeline produces 43 fully voiced lines in 19.4 minutes on a 4 GB RTX 2050. Both goals are recovered with precision and recall 1.00, but overall event recall against ten curated moments is 0.40 at ±1 minute. On 40 held-out human references, GPT-2 v4 improves ROUGE-L from 0.093 to 0.185 and METEOR from 0.027 to 0.207 over templates, while BLEU-4 remains near zero. A 5-minute causal chunk is faster to first line (4.8 vs 16.6 minutes) but misattributes both goals to the home side, so the current system is pre-recorded only. Cloud inference costs $0.39 per match versus an illustrative $300 human team. We conclude that low-cost, auditable pre-recorded commentary is feasible; live commentary and general event spotting remain open.

Keywords: football commentary, action spotting, multi-object tracking, LoRA, grounding, text-to-speech, reproducibility

1. Introduction

A football broadcast is expensive to describe. A live commentator, a producer and often a second analyst must watch every match, identify the important moments, remember the score and the narrative, and speak continuously. Most matches are never covered at all; the ones that are covered are described in one language for one audience. Automatic commentary could widen that coverage and lower its cost, but only if it is accurate enough to trust and cheap enough to run.

Three families of systems have been proposed. Rule-based and template systems such as MIKE separate fact selection from surface realisation and remain interpretable, but their language is repetitive [6]. Neural captioning systems, including SoccerNet-Caption and MatchTime, learn from broadcast commentary and produce more varied text, but depend on large aligned corpora and report overlap or alignment metrics rather than factuality [7], [8]. More recent multimodal LLMs such as TimeSoccer and SoccerComment consume full matches or retrieve external knowledge, but they are heavy and do not expose an auditable fact layer [9], [10]. Each family trades one desirable property for another.

This paper asks whether a deliberately small, modular system can occupy a different point in that trade-off. Matchcall keeps five stages separate: event spotting, tracking and team/possession facts,

deterministic description, language generation, and speech. The language model never decides the score, minute, team or possession; it only realises facts that were already measured. A validator checks every candidate line against those facts, and a grounded template is used when the candidate fails. The design target is a full match on a single consumer GPU, with every training run and artifact logged.

We report four comparisons on a real broadcast: (i) GPT-2 commentary against 40 held-out human references and a template baseline; (ii) pre-recorded full-context commentary against a 5-minute causal chunk; (iii) event spotting against ten curated reference moments; and (iv) compute, TTS and human cost. The results are deliberately mixed. Goals are reliable because the broadcast score bug is a high-precision anchor. General event spotting and live latency are not solved. The contribution is not a new state of the art on SoccerNet; it is an auditable, reproducible instantiation of the full broadcast-to-speech pipeline, with its failure modes measured rather than hidden.

Contributions. (1) A modular five-stage pipeline that separates deterministic facts from language and speech. (2) A score-bug goal anchor that infers the on-screen starting score and survives short clips. (3) A full 64.5-minute end-to-end run with 43 voiced lines on a 4 GB GPU. (4) Four empirical comparisons, including negative results for live operation and general event spotting. (5) ClearML-logged training runs and a reproducible evaluation script.

2. Related Work

2.1 Action spotting and event understanding

SoccerNet defined action spotting as predicting the temporal location and class of sparse events in long broadcasts, with Average-mAP at multiple tolerances [1]. SoccerNet-v2 expanded the taxonomy to seventeen classes and added camera-shot and replay annotations [2]. Later work improves temporal context and timestamp precision, but the common lesson is that temporal precision and a large, well-annotated corpus matter. Our system instead uses a broadcast-specific shortcut for goals — the score bug — and a lightweight window classifier for other events. The trade-off is lower coverage on rare classes, which we measure rather than assume away.

2.2 Detection, tracking, team identification and possession

YOLO established single-pass real-time detection [21]. ByteTrack associates low-score boxes to recover occluded objects and is the tracker used here [3]. Team identity and possession are semantic quantities that track IDs alone do not provide. DeepPlayer-Track uses jersey-colour masks for team recognition [4], and Guo et al. combine YOLOv8, DBSCAN jersey clustering and Norfair tracking for possession estimation on full matches [5]. Our pipeline follows the label-free jersey-clustering approach and adds a bounded ball-interpolation rule; the ball remains a weak point because its object-detection AP is low.

2.3 From templates to neural and LLM commentary

MIKE used six analysers, importance-scored proposition selection and template generation [6]. SoccerNet-Caption introduced dense captioning with almost 37,000 timestamped commentaries across 715.9 hours [7]. MatchTime showed that alignment quality is a first-order determinant of generation quality and released an aligned dataset and the MatchVoice model [8]. TimeSoccer consumes full 45-minute matches with frame compression [9], while SoccerComment combines LanguageBind, Vicuna-7B and retrieval [10]. MatchVision/SoccerReplay-1988 provides a large

multi-modal corpus [23]. These systems are more fluent than templates but heavier; our work tests a small GPT-2 with LoRA [11], [12] under a deterministic validator.

2.4 Grounding, hallucination and evaluation

Hallucination in natural-language generation is surveyed by Ji et al. [14]. FActScore decomposes long-form text into atomic facts and checks each against a knowledge source [13]. Football commentary has no dedicated faithfulness benchmark. SoccerNet-Caption, MatchTime, TimeSoccer and SoccerComment report captioning overlap or alignment; BLEU, ROUGE and METEOR measure similarity to references, not truth [18]–[20]. Our validator is closer to atomic checking than to retrieval-only grounding: it checks the generated line against the measured score, minute, team and event before acceptance.

2.5 Speech synthesis and prosody

Tacotron 2 established neural TTS naturalness [15]; FastSpeech 2 added explicit duration, pitch and energy control [16]. Fish Audio's S1 family offers emotion and tone markers and reports low real-time factors on high-end GPUs [17]. Sports-commentary prosody is not covered by these benchmarks. We use a production TTS and map event intensity to pacing, but we do not claim a sports-specific TTS evaluation.

2.6 Positioning

The literature is rich in perception benchmarks and increasingly rich in generative commentary, but sparse in systems that expose a deterministic fact layer, run the full broadcast-to-speech path on consumer hardware, and report their negative results. Matchcall is positioned at that intersection. It borrows the score-bug idea for goals, the jersey-clustering idea for teams, the LoRA idea for small-GPU language, and the FActScore idea for validation; it combines them into one auditable pipeline and measures where the combination fails.

3. System

3.1 Overview

Matchcall has five stages, shown in Figure 1. The first two produce observations; the third turns them into facts; the fourth turns facts into language; the fifth turns language into speech and mixes it with the match. The separation is deliberate: an error in the language model cannot change the score, and an error in perception is visible in the fact sheet before it reaches the viewer.



Figure 1. The five-stage pipeline. Stages 1–2 observe; stage 3 fixes the facts; stage 4 realises them; stage 5 voices and renders. The language model never writes the score or the event team.

3.2 Event layer

The event layer combines three signals. A score-bug reader samples the two score digits in the broadcast overlay, classifies each digit by its binary shape, and detects a persistent transition to the expected next score. It infers the score already on the board from the first three samples, so a clip that starts at 2–1 is not treated as 0–0. A motion-gated window scan proposes candidate events, and a ResNet-18 classifier assigns one of five labels (goal, shot, pass, tackle, dead time); crowd-audio rise and ball speed refine the goal/shot decision. The score-bug anchor is treated as gold for goals: it is placed a few seconds before the digit flip because the broadcast updates after the ball crosses the line.

3.3 Tracking, teams and facts

YOLO26n detects players, the ball and the referee in each scheduled frame; ByteTrack maintains identities across frames [3], [21]. Jersey colours are clustered with KMeans into two teams, and possession is assigned to the side of the tracked player nearest the ball. Because broadcast cuts and close-ups break the ball path, raw detections are interpolated only across bounded gaps (16 frames), single-frame teleports are rejected, and predicted frames are marked. The output is a deterministic fact sheet: score, minute, ball zone, possession, tracked-player count and event team. No language model is allowed to add to this sheet.

3.4 Commentary and validation

GPT-2 v4 is a LoRA adapter [11] over GPT-2 [12], trained on team-grounded SoccerReplay commentary [23]. The prompt supplies the event label, team, scoreline, minute, phase and measured vision features. Candidate lines are cleaned and checked: the event team must appear, blocked team and player names are rejected, digits must match the fact sheet, and a goal line must contain a real

goal phrase in its opening sentence and no contradiction such as “denied,” “wide” or “saved.” If every candidate fails, the grounded template announcer is used [6]. The validator is intentionally conservative; it is a guard against invented facts, not a measure of fluency.

3.5 Speech, render and MLOps

Accepted lines are synthesised with Fish Audio s2.1-pro-free [17]. Event intensity controls pacing, the voice is mixed under the original crowd at a low level, and ffmpeg renders the full match. Every YOLO and GPT-2 run is a ClearML task with parameters, per-epoch or per-step scalars, plots, artifacts and a registered model [22]. The evaluation artifacts in this paper were produced from the same logged runs.

4. Experimental Setup

4.1 Data and tasks

All end-to-end results use a 64.5-minute, 1920×1080 recording of the Argentina–France second half. The recording contains two goals, at broadcast minutes 80 and 81; the score bug flips from 2–1 to 2–2. The event reference is the ten curated moments used by the earlier highlight reel: six shots, two tackles and two goals. This is a reference for recall/consistency, not an exhaustive independent annotation, so event precision is not measured. The commentary comparison uses 40 held-out SoccerReplay prompts with human reference lines [23]. The live experiment uses a 5-minute chunk starting at 40:00 of the recording, processed with only that chunk as context. The object detector is trained on the Roboflow V1 football dataset (8,677 train / 2,996 validation images, three classes: Player, Ball, Referee) [24]. These are the only datasets used for training in this paper; SoccerNet and other benchmarks appear only as related work.

4.2 Models and training

Table 1 summarises the trained components. YOLO26n was fine-tuned at 640 px for 50 epochs with batch 1 on the RTX 2050; the best checkpoint was the final epoch, with mAP50 0.806 and mAP50–95 0.480. GPT-2 v4 used LoRA rank 32, 160-token sequences, completion-only loss, three epochs and 2,481 optimizer steps; the loss fell from 3.877 to 0.779. Fish Audio was used through its free tier, so no paid TTS price is included.

Table 1. Trained components and schedules. The detector and language model were trained locally on one 4 GB GPU; ClearML logged both runs.

Component	Model / data	Params	Schedule	Result
Detector	YOLO26n on V1 (8,677 / 2,996)	2.39 M	640 px, 50 epochs, batch 1, 2.70 h	mAP50 0.806; mAP50–95 0.480
Commentary	GPT-2 + LoRA r=32 on SoccerReplay-1988	124 M base	3 epochs, 2,481 steps, 160 tokens	loss 3.877 → 0.779
Speech	Fish Audio s2.1-pro-free	—	43 calls, 412.2 s speech	43/43 lines voiced

4.3 Metrics

Commentary is scored against human references with BLEU-1/2/4, ROUGE-L and METEOR [18]–[20], and with validator acceptance and line length. Event spotting is scored as recall against the ten reference moments at ±1 minute, with goal precision/recall checked against the known 80' and 81' goals. Live operation is measured by wall time, wall-time ratio and time to first commentary line. Cost is reported for local electricity, a cloud ml.g5.2xlarge at $1.21/h, and an illustrative two-person human team; the human rates are assumptions, not quotes.

4.4 Hardware and reproducibility

Inference and training used an NVIDIA RTX 2050 with 4 GB of VRAM. The full-match run, ClearML tasks, model checkpoints, comparison JSON and evaluation scripts are retained in the project repository. The key commands are listed in Appendix A.

5. Results

5.1 AI versus human commentary

Table 2 reports the 40-prompt comparison. GPT-2 v4 more than doubles ROUGE-L over the template baseline (0.185 vs 0.093) and raises METEOR from 0.027 to 0.207, while writing lines of human length (40.5 words vs 31.9 for humans). BLEU-4 is near zero for both systems, which is expected for free-form commentary with many valid paraphrases. The validator accepts 38 of 40 lines; an independent held-out evaluation with a stricter acceptance path accepted 15 of 40, so the 95% figure should be read as a property of this prompt set, not a general truth rate.

Table 2. Commentary against 40 held-out human references. The human column is the target; self-similarity is 1 and is not a system score.

Metric	GPT-2 v4	Template	Human reference
BLEU-1	0.0300	0.0103	—
BLEU-2	0.0014	0.0020	—
BLEU-4	0.000304	0.001038	—
ROUGE-L	0.1845	0.0931	—
METEOR	0.2066	0.0269	—
Average words	40.5	7.4	31.9
Grounding accepted	38/40	—	—

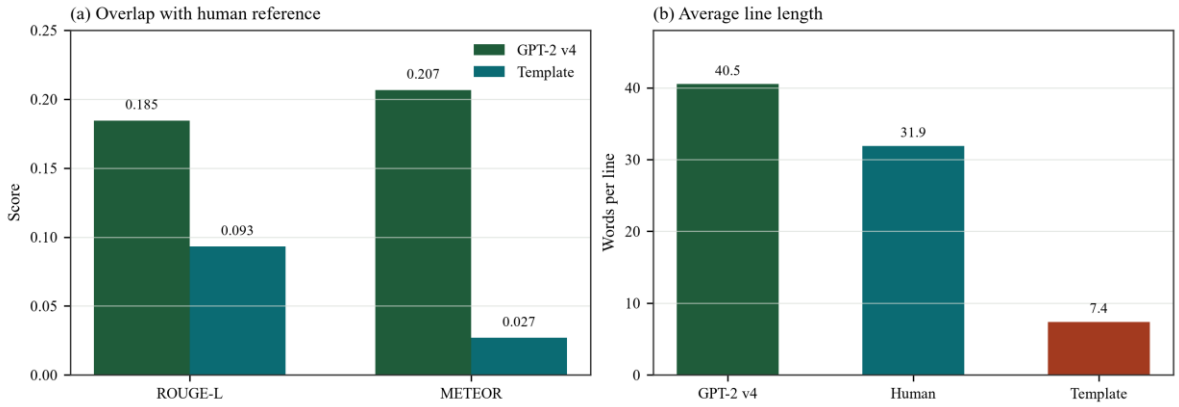


Figure 2. GPT-2 v4 is closer to human overlap and length than the template baseline. BLEU-4 is omitted because it is near zero for both systems; exact values are in Table 2.

Table 3 quotes three held-out records to show what the aggregate scores hide. In the first, GPT-2 names the correct team and produces a human-length line, but invents a shot that the reference does not contain. In the second, both GPT-2 and the template attribute the corner to Napoli while the event belongs to Eintracht Frankfurt; this is a shared feature-layer failure, not only a language failure. In the third, GPT-2 misses a goal entirely and describes a stop and a corner; the template at least calls a goal, but names the wrong team. These correspond to the three failure modes: invented detail, wrong team, missed event.

Table 3. Held-out commentary records. Pink highlights mark wrong or unsupported content; green highlights mark a correct event call.

Event	Source	Line	Difference
Record A · pass / West Ham · human and GPT-2 both describe the same passage			
pass	Human	Wonderful movement by West Ham. The players are holding possession, mostly exchanging one-touch passes. They control the pace of the game now. The midfielder attempts to feed the defender with a chip pass, but the goalkeeper reacts well and comes off his line to intercept.	Reference line
pass	GPT-2 v4	West Ham are showcasing great team work by exchanging some short passes and one-two passing balls with one another to create new attacking opportunities. The midfielder is unlucky enough not to shoot from the edge of the box as his effort flies just wide of the right post.	invented shot; team correct
pass	Template	Patient stuff from West Ham, switching the play smartly.	correct but short
Record B · corner / Eintracht Frankfurt · both systems name the wrong team			
corner	Human	The forward strikes the ball towards goal, but it fails to reach the target after being well blocked by one of the defenders. Corner kick. Eintracht Frankfurt will have an opportunity.	Reference line
corner	GPT-2 v4	Napoli failed to take advantage of the corner as their opponents were alert and averted the threat at the last moment! The ball goes out of play. Neat result from the forward's attempted effort with a cross into the box resulting in a dangerous clearance.	wrong team; topic correct
corner	Template	The delivery comes in for Napoli!	wrong team
Record C · goal / B. Monchengladbach · GPT-2 misses the event			
goal	Human	The goal stands! That took a long time for the referee to decide. The relieved B. Monchengladbach players can finally celebrate.	Reference line
goal	GPT-2 v4	The forward races towards goal but the defender gets back well to make a challenge. One of the defending players reacts superbly to get his teammates down and intercepts one. A promising stop for Inter! Corner kick from long range.	missed goal; wrong team
goal	Template	GOAL! Inter make it count — sheer precision at a vital time!	calls goal; wrong team

Result. GPT-2 is the better surface realiser: longer, more varied, closer to human overlap. It is not the better fact layer. The validator raises precision but accepts 95% of lines, and in one record both the model and the template inherit a wrong team from the shared feature layer.

5.2 Live versus pre-recorded

The full-context run is a batch job: it reads the whole match, detects events, writes lines and only then synthesises speech. The 5-minute chunk is processed with only that chunk as context and is therefore a closer analogue of a causal live system. Table 4 shows the difference. The full run calls both goals correctly and attributes them to France. The chunk calls two goals and attributes both to Argentina, while the score-bug anchors in the same run are away 44:33 and away 45:42; it also produces an earlier goal at 2:01. The chunk is faster to first line (4.8 vs 16.6 minutes) but factually worse.

Table 4. Full-context versus 5-minute causal chunk. Pink marks the wrong side; green marks the correct side.

Mode	First line	Goal commentary	Outcome
Pre-recorded, full context	16.6 min	[44:18] GOAL! France make it count — sheer precision at a vital time! [45:27] IT'S A GOAL! Clinical from France — the keeper had no chance!	2/2 goals; correct side; slower
Live-like, 5-min chunk	4.8 min	[02:01] GOAL! Argentina make it count — sheer precision at a vital time! [04:05] IT'S A GOAL! Clinical from Argentina — the keeper had no chance!	wrong side; false early goal; faster

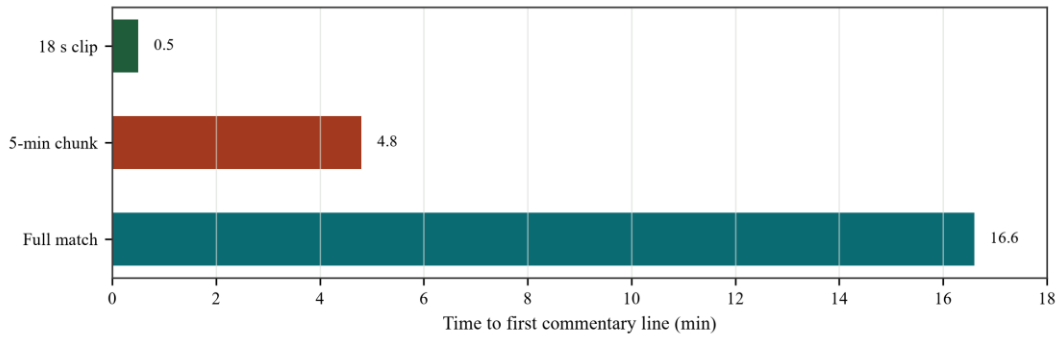


Figure 3. Latency to the first commentary line. The full-match run is faster than real time in aggregate (0.30×), but the first line appears only after the whole match is scored and written. The chunk is near real time and wrong.

The chunk's failure has a concrete cause: the goal-side information exists in the anchor, but it is not carried into the realised line, and the short context makes the event classifier over-fire. A live system needs streaming perception, incremental TTS and explicit side propagation; shortening the input is not enough. This is a negative result for the current architecture, and we report it as such.

5.3 Event spotting

Figure 4 compares the ten reference moments with the 38 event lines from the full run. Figure 5 summarises recall. Goals are the strong case: both are found within 13–22 seconds, with precision and recall 1.00. Shots are weaker: two of six are matched, with a timing error of 51–59 seconds. Both tackles are missed. Overall recall is 0.40 at ±1 minute. The pipeline's label distribution explains part of the gap: it emits 19 pass and 7 dead-time lines, but only 5 shot, 5 corner and 2 goal lines.

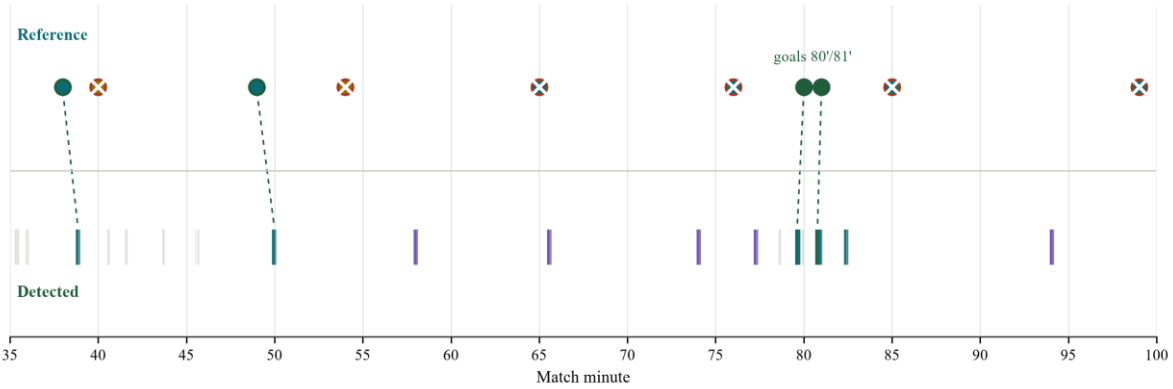


Figure 4. Reference moments (top) and detected event lines (bottom). Faint ticks are passes and dead time; strong ticks are goals, shots, corners and tackles. Dashed lines mark matches within ±1 minute; red-ringed markers mark missed reference events.

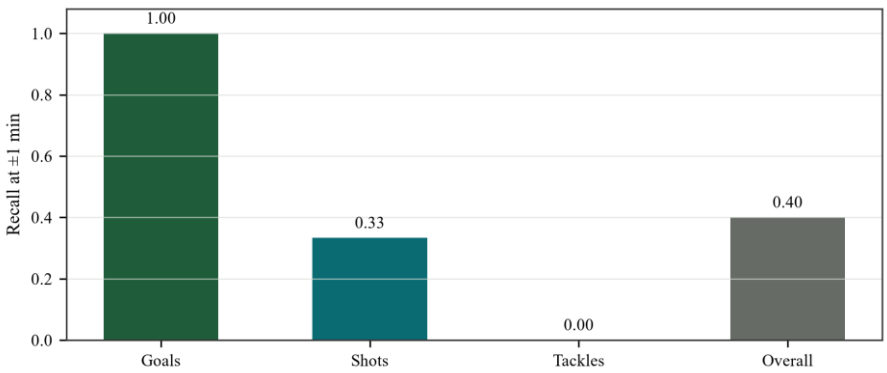


Figure 5. Event recall against the ten curated moments. Goal precision and recall are both 1.00; timing MAE for matched events is 36.2 s.



Figure 6. The 81' goal clip. The broadcast score flips from 2–1 to 2–2; the score-bug reader infers the starting score and anchors the away goal at 5 s. Frames at 0, 3, 6, 9, 12 and 15 s.

Table 5. Reference moments versus detected events at ±1 minute. A dash means no same-kind detection in the window.

Minute	Reference	Detected	Result
38	shot	38.85 shot	matched (+50.8 s)
40	tackle	—	missed
49	shot	49.98 shot	matched (+58.8 s)
54	tackle	—	missed
65	shot	65.56 corner	missed
76	shot	—	missed
80	goal	79.63 goal	matched (−22.2 s)
81	goal	80.78 goal	matched (−13.2 s)
85	shot	—	missed
99	shot	—	missed

Object detection is a separate measurement. Table 6 compares YOLO26n with the paused YOLOv5lu run. YOLO26n is fractionally ahead in aggregate mAP, but the budgets are not equal: 50 vs 33 epochs and 2.39 M vs 53 M parameters. Per-class YOLO26n mAP50–95 is 0.621 for Player, 0.291 for Ball and 0.530 for Referee. The low Ball score is why the ball path is interpolated rather than trusted, and it is a plausible contributor to the missed shot/tackle events.

Table 6. Object detection on the V1 validation split. This is not event-spotting accuracy.

Model	Params	Epochs	mAP50	mAP50–95	Precision	Recall
YOLO26n	2.39 M	50	0.805	0.481	0.864	0.763
YOLOv5lu	53 M	33	0.795	0.459	0.854	0.752

Result. The score-bug goal path is reliable enough to build on. The general event classifier is not: it finds one third of reference shots, no tackles, and emits more pass/dead-time lines than event lines. Detection mAP must not be reported as event-spotting accuracy.

5.4 Training dynamics

Figure 7 shows the YOLO26n training and validation curves. The box and classification losses fall steadily, and mAP50 rises from 0.538 at epoch 1 to 0.806 at epoch 50; mAP50–95 follows from 0.269 to 0.480. The curve is still improving at the final epoch, so 50 epochs is a budget choice rather than a converged optimum. Figure 8 shows the GPT-2 LoRA loss: a rapid fall from 3.877 to about 1.0 in the first epoch, then a slow, noisy decline to 0.779 at step 2,481. The final held-out acceptance is not a monotone function of training loss; language quality and factual grounding are different objectives.

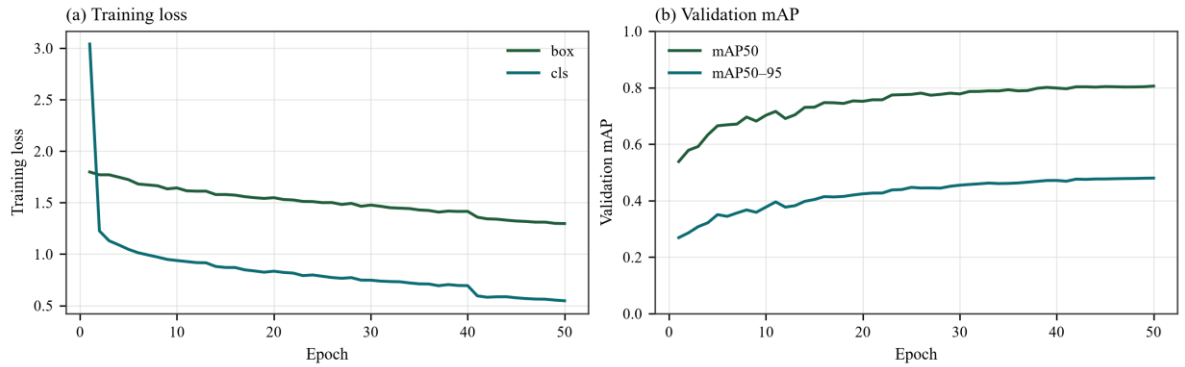


Figure 7. YOLO26n training curves, 640 px and 50 epochs. mAP50 reaches 0.806 and mAP50–95 0.480 at the final epoch; the model is still improving, so the schedule is a compute constraint, not a converged optimum.

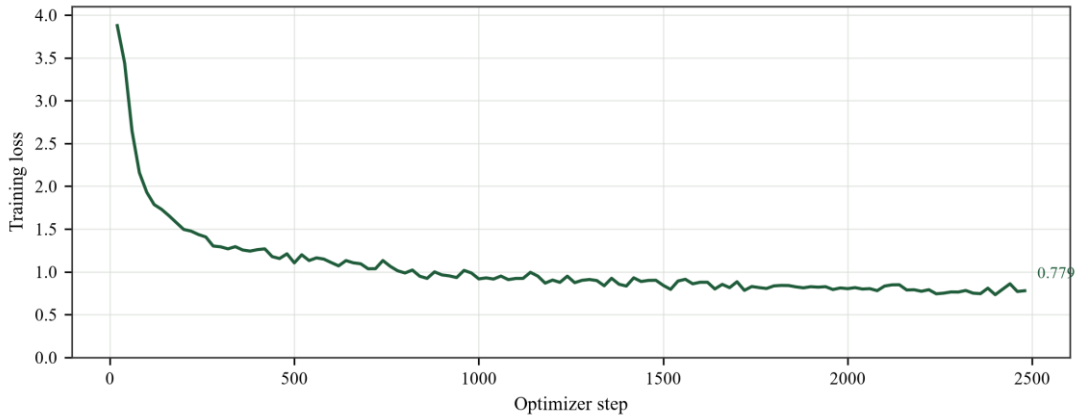


Figure 8. GPT-2 v4 LoRA training loss over 2,481 steps. The rapid first-epoch drop is followed by a slow, noisy decline; the final loss is 0.779.

5.5 Cost

Table 7 reports the measured compute and an illustrative human baseline. A full match costs about one cent of local electricity or $0.39 on a cloud A10G. Training both models for 3.64 hours costs $0.109 locally or $4.40 at the cloud rate. The human scenarios are illustrative: a two-person team at $50/h for three live hours or four pre-recorded hours costs $300 or $400. Across the $25–$100/h range, cloud compute is roughly 400–1,500× cheaper. The caveat is that compute is not the only cost: latency, quality review and paid TTS are excluded.

Table 7. Cost per 64.5-minute match. Local electricity and cloud rates are measured/assumed; human rates are illustrative, not quotes.

Item	Local RTX 2050	Cloud g5.2xlarge	Human team
Inference	$0.010	$0.392	$300 live / $400 pre-recorded
Training (both models)	$0.109	$4.40	—
Training hours	3.64 h	3.64 h	—
TTS calls / speech	43 / 412.2 s	43 / 412.2 s	—

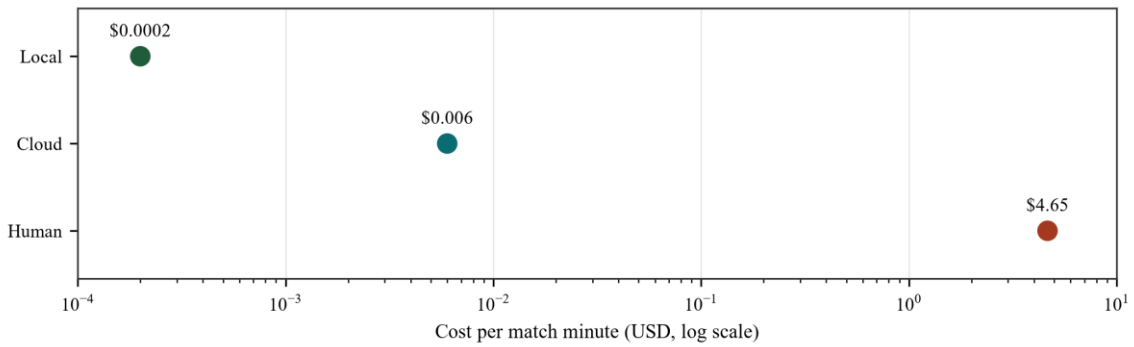


Figure 9. Cost per match minute on a log scale. The local and cloud points are two to three orders of magnitude below the human scenario; the barrier to deployment is quality and latency, not compute.

6. Discussion

The experiments support a narrower claim than "automatic commentary." The pipeline is a cheap, auditable, pre-recorded system whose goal path is reliable and whose general event layer is not yet good enough for broadcast output. Three findings organise the rest.

Score-bug anchoring is the strongest component. A persistent digit flip is a high-precision signal that survives replays, close-ups and celebration cuts, and it names the scoring side. The full-match run finds both goals with precision and recall 1.00. The main engineering requirement is to carry the anchored side into the realised line; the 5-minute chunk shows what happens when that link is missing.

Language quality and factuality are separate objectives. GPT-2 v4 more than doubles ROUGE-L over templates and writes human-length lines, but it invents shots, misnames teams and can miss a goal. The validator improves precision but accepts 95% of lines on this prompt set; it checks words, not truth. A template fallback is safer but not sufficient when the feature layer itself carries the wrong team.

Live operation is an architectural problem, not a speed knob. The full run is faster than real time in aggregate, but the first line appears after 16.6 minutes. The causal chunk is faster but factually worse. A live system needs streaming perception, incremental speech and explicit side propagation; the current batch architecture should be described as pre-recorded.

For the paper's framing, the strongest contribution is the separation of deterministic facts from language and the score-bug goal path. The weakest links are general event spotting, team-side propagation and live latency. Reporting those weaknesses is more useful than claiming another state-of-the-art commentary model.

7. Threats to Validity

Reference validity. The ten event moments are curated from the earlier highlight reel, not an exhaustive independent annotation. The 0.40 recall is a consistency check against those moments; event precision for shots, tackles and corners is not measured. The 2/2 goal result is exact for this recording but has only two samples.

External validity. All end-to-end results come from one 64.5-minute broadcast, one competition and one production style. The score-bug boxes are tuned to this overlay; another broadcaster would need recalibration. The detector was trained on a small Roboflow export.

Construct validity. BLEU, ROUGE and METEOR measure overlap, not truth or entertainment. The validator acceptance rate is not a human preference score. No human evaluation of commentary quality was run; the paper reports the need for one.

Budget validity. YOLO26n and YOLOv5lu did not receive equal epochs or parameter budgets, so the mAP comparison is descriptive, not a controlled model comparison. The human cost is illustrative; paid TTS, storage and egress are excluded.

Reproducibility. The pipeline is deterministic given the same video and model weights, but the generative language model samples; lines vary across runs. The reported commentary examples are single samples.

8. Conclusion

Matchcall shows that a small, modular, fact-grounded pipeline can turn a full broadcast into voiced commentary on a consumer GPU at a cost far below a human team. The goal path works because the score bug is an explicit, high-precision signal; the language path works well enough to produce human-length lines; the cost is negligible. The system does not yet work as live commentary, does not find most non-goal events, and can attribute a goal to the wrong team. Those are the next problems: streaming architecture, side propagation, better event annotation and a human evaluation of commentary quality. The broader lesson is that separating facts from language makes both the successes and the failures measurable.

References

[1] S. Giancola, M. Amine, T. Dghaily, and B. Ghanem, “SoccerNet: A scalable dataset for action spotting in soccer videos,” in Proc. CVPR Workshops, 2018, pp. 1711–1721.

[2] A. Deliège, A. Cioppa, S. Giancola, et al., “SoccerNet-v2: A dataset and benchmarks for holistic understanding of broadcast soccer videos,” in Proc. CVPR Workshops, 2021, pp. 4508–4519.

[3] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-object tracking by associating every detection box,” in Proc. ECCV, 2022.

[4] R. Naik, M. Hashmi, Z. Geem, and N. Bokde, “DeepPlayer-Track: Player and referee tracking with jersey color recognition in soccer,” IEEE Access, vol. 10, pp. 32494–32509, 2022.

[5] Y. Guo, H. Zeng, X. Deng, et al., “Automatic estimation of football possession via improved YOLOv8 detection and DBSCAN-based team classification,” Sensors, vol. 26, no. 4, 1252, 2026.

[6] K. Tanaka-Ishii, S. Noda, I. Frank, et al., “MIKE: An automatic commentary system for soccer,” in Proc. ICMAS, 1998, pp. 285–292.

[7] H. Mkhallati, A. Cioppa, S. Giancola, B. Ghanem, and M. Van Droogenbroeck, “SoccerNet-Caption: Dense video captioning for soccer broadcasts commentaries,” in Proc. CVPR Workshops, 2023, pp. 5074–5085.

[8] J. Rao, J. Wu, Z. Liu, et al., “MatchTime: Towards automatic soccer game commentary generation,” in Proc. EMNLP, 2024, pp. 1671–1685.

[9] X. You, J. Huang, Y. Xie, et al., “TimeSoccer: An end-to-end multimodal large language model for soccer commentary generation,” in Proc. ACM Multimedia, 2025, pp. 3418–3427.

[10] Y. Li, J. He, X. Zu, et al., “Multi-modal large language model with RAG strategies in soccer commentary generation,” in Proc. WACV, 2025, pp. 6197–6206.

[11] E. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-rank adaptation of large language models,” in Proc. ICLR, 2022.

[12] A. Radford, J. Wu, R. Child, et al., “Language models are unsupervised multitask learners,” OpenAI technical report, 2019.

[13] S. Min, K. Krishna, X. Lyu, et al., “FActScore: Fine-grained atomic evaluation of factual precision in long-form text generation,” in Proc. EMNLP, 2023, pp. 12076–12100.

[14] Z. Ji, N. Lee, R. Frieske, et al., “Survey of hallucination in natural language generation,” ACM Computing Surveys, vol. 55, no. 12, pp. 1–38, 2023.

[15] J. Shen, R. Pang, R. Weiss, et al., “Natural TTS synthesis by conditioning WaveNet on mel spectrogram predictions,” in Proc. ICASSP, 2018.

[16] Y. Ren, C. Hu, X. Tan, et al., “FastSpeech 2: Fast and high-quality end-to-end text to speech,” in Proc. ICLR, 2021.

[17] Fish Audio, “Launching Fish Audio S1: A frontier text-to-speech audio foundation model,” vendor blog, 2025.

[18] K. Papineni, S. Roukos, T. Ward, and W. Zhu, “BLEU: A method for automatic evaluation of machine translation,” in Proc. ACL, 2002, pp. 311–318.

[19] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in Proc. ACL Workshop, 2004, pp. 74–81.

[20] S. Banerjee and A. Lavie, “METEOR: An automatic metric for MT evaluation with improved correlation with human judgments,” in Proc. ACL Workshop, 2005, pp. 65–72.

[21] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in Proc. CVPR, 2016, pp. 779–788.

[22] ClearML, “Task and experiment tracking documentation,” clear.ml/docs, 2025.

[23] J. Rao, J. Wu, B. Zhang, et al., “Towards universal soccer video understanding,” in Proc. CVPR, 2025, pp. 8384–8394.

[24] Roboflow Universe, “Football-players-detection dataset,” 2023. [Online]. Available: https://universe.roboflow.com/roboflow-uc1d1/football-players-detection

Appendix A. Reproducibility

The main artifacts are produced by the following commands. The full-match job is 7c23a8a91829; the corrected goal clip is 8cdba651f954; the 5-minute chunk is feebe1ed5d60.

```
python scripts/compare_commentary.py --n 40 --device cuda
python scripts/evaluate_event_spotting.py
python scripts/compile_experiments.py
```

Data and outputs: output/comparison/commentary_comparison.json; output/experiments/event_spotting.json; output/experiments/experiment_results.json; output/experiments/full_match_job.json; output/experiments/chunk_job.json. Models: models/yolo26/yolo26n_v1_640; models/gpt2_lora_v4. Commit 7f1a826.